0046. Permutations

Medium | Category: Backtracking | LeetCode: https://leetcode.com/problems/per

Problem

- Return all permutations of an array of distinct integers.

Intuition

- Build the permutation position by position; at each step choose any unused element.
- A boolean used[] array tracks what is already in the path.

Brute Force

- Generate all n^n sequences and filter those with distinct elements.

Optimal Approach

- If path.size() == n, record.
- For each i not used: mark, add, recurse, unmark, remove.

Worked Example

```
[1,2,3].

1 -> 2 -> 3   [1,2,3]
  -> 3 -> 2   [1,3,2]
2 -> 1 -> 3 ; 2 -> 3 -> 1
3 -> 1 -> 2 ; 3 -> 2 -> 1
6 permutations
```

Java Solution

```java
public class Permutations {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        dfs(nums, new boolean[nums.length], new ArrayList<>(), res);
        return res;
    }

    private void dfs(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> res) {
        if (path.size() == nums.length) { res.add(new ArrayList<>(path)); return; }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true;
            path.add(nums[i]);
            dfs(nums, used, path, res);
            path.remove(path.size() - 1);
            used[i] = false;
        }
    }
}
```

Complexity

- Time: O(n n!). Space: O(n).

Pitfalls

- Permutations iterate from 0 each time (order matters); subsets/combinations iterate from start.